Практика 7. Анализ предупреждения статического анализатора

Файл: BAD_SIZEOF.pdf, проект server, snapshot 2024-03-26

1. BAD_SIZEOF

Isolated occurrences of 'sizeof' operators are technically legal in C/C++, yet are often erroneous.

Language	Severity	Reliability	Status	Situation
CXX	Critical	Average	true	Quality

Location: /server/sql/lock.cc:802	Undecided
Function	MYSQL_LOCK * get_lock_data(THD *, TABLE **, uint, uint)
Warning message	use of sizeof on a function parameter 'table_ptr' which has 'struct TABLE **' pointer type

```
797    update the table values. So the second part of the array
       is copied
798    from the first part immediately before calling
       thr_multi_lock().
799  */
800  size_t amount= sizeof(*sql_lock) +
801                 sizeof(THR_LOCK_DATA*) * lock_count * 2 +
802                 sizeof(table_ptr) * table_count;
803  if (!(sql_lock= (MYSQL_LOCK*) (flags & GET_LOCK_ON_THD ?
804                                 thd->alloc(amount) :
805                                 my_malloc(key_memory_MYSQL_LO
                                    CK, amount,
806                                           MYF(0)))))
807    DBUG_RETURN(0);
```

Location: /server/sql/sql_acl.cc:9933	Undecided
Function	bool show_table_and_column_privileges(THD *, const char *, const char *, char *, size_t)
Warning message	use of sizeof on a function parameter 'buff' which has 'char *' pointer type

```
9928    cols_access= grant_table->init_cols;
9929  }
9930
9931  if ((table_access | cols_access) != NO_ACL)
9932  {
9933    String global(buff, sizeof(buff), system_charset_info);
9934    privilege_t test_access= (table_access | cols_access) &
       ~GRANT_ACL;
9935
9936    global.length(0);
```


Решение по файлу: Won't fix (ложное срабатывание)

Колесников Андрей Юрьевич, 4 курс, группа БСБО-03-23

Студенческий билет 23Б0099, номер по журналу 10

Комментарий

Предупреждение BAD_SIZEOF в точке /server/sql/lock.cc:802 относится к строке sizeof(table_ptr) * table_count. Формально Svace прав: table_ptr — это параметр функции с типом TABLE **, поэтому

sizeof(table_ptr) возвращает размер указателя (8 байт на x86_64), а не размер структуры TABLE. Однако в данном выражении выделяется память под массив указателей TABLE*, а не под массив структур. Размер элемента такого массива — тоже указатель, и на целевой платформе sizeof(TABLE*) совпадает с sizeof(TABLE**). Поэтому amount вычисляется верно, недостатка памяти и переполнения кучи здесь нет.

Предупреждение возникло из-за того, что анализатор видит изолированный sizeof от параметра-указателя и не учитывает, что рядом выделяется массив указателей. Это контекстно-зависимый случай: если бы table_ptr имел тип TABLE* и массив состоял из структур TABLE, тогда sizeof(table_ptr) действительно привёл бы к недостаточного выделения памяти и переполнению. В текущем коде этого нет.

Эксплуатация невозможна: значение table_count не влияет на размер элемента, выделение соответствует дальнейшему копированию массива указателей. Отмечаю только стилистическую рекомендацию: заменить sizeof(table_ptr) на sizeof(*table_ptr) — результат тот же, но намерение видно сразу и код не будет ломаться при будущем изменении типа параметра. Это не исправление уязвимости, поэтому статус файла — Won't fix.

Второе срабатывание в этом же файле, /server/sql/sql_acl.cc:9933, относится к другому участку: там sizeof(buff) для параметра char * действительно даёт 8, а не размер буфера. Оно не входит в выбранную точку lock.cc:802 и требует отдельного разбора; в отчёте по взятому участку оно не учитывается.

Итог: предупреждение в lock.cc:802 — ложное срабатывание, ошибки памяти нет. Решение: Won't fix, с комментарием о желательной замене на sizeof(*table_ptr) для читаемости.